



BITS Pilani
K K Birla Goa Campus

INSTRUCTION SET WITH MASM (INT 10H)

MICROPROCESSORS & INTERFACING (CS F241)

Dr. Gargi Alavani
Department of CS & IS



Who set the ISR locations?

During boot:

- BIOS loads and initializes hardware
- BIOS fills IVT entries with ISR addresses
- Those addresses point to BIOS routines in ROM
- So BIOS decides the actual locations.

Because 8086 runs in **real mode with no protection**:

- ✓ Programmers could read IVT entries
- ✓ They could modify them
- ✓ They could install their own ISR



Interrupt hooking is a technique that intercepts and modifies the execution flow of system interrupts by replacing or chaining into the **Interrupt Service Routine (ISR)** table.

Interrupt Control

- The set interrupt flag instruction (**STI**) places a 1 into the I flag bit, which **enables the INTR pin**.
- The clear interrupt flag instruction (**CLI**) places a 0 into the I flag bit, which **disables the INTR pin**.
- In a software interrupt service procedure, hardware interrupts are enabled as one of the first steps.
- This is accomplished by the STI instruction.
- The reason interrupts are enabled early in an interrupt service procedure is all of the I/O devices in the personal computer are interrupt-processed.
- If the interrupts are disabled too long, severe system problems result.

Enables interrupt nesting

- ISR A is running
- A new hardware interrupt occurs
- CPU temporarily pauses ISR A
- Runs ISR B (higher priority)
- Returns to ISR A afterward

Enables interrupt nesting

- ISR A is running
- A new hardware interrupt occurs
- CPU temporarily pauses ISR A
- Runs ISR B (higher priority)
- Returns to ISR A afterward

Software interrupts (e.g., system calls, OS services) often:

- Perform longer operations
- Are less urgent than hardware events

Controlling the Carry Flag Bit

- The carry flag (C) propagates the carry or borrow in multiple-word/doubleword addition and subtraction.
- It also can indicate errors in assembly language procedures.
- Three instructions control the contents of the carry flag:
 - **STC** (set carry)
 - **CLC** (clear carry)
 - **CMC** (complement carry)

Int 10H

- **INT 10H** subroutines are burned into ROM BIOS and are used to communicate with the **computer's screen video**
- Provides a set of functions to control the video display such as setting the video mode, moving the cursor, scrolling the screen, displaying characters, and more
- Serves as an interface between the software (assembly language programs) and the video hardware of the computer
- Chosen by putting a specific value in register AH

Modes

INT 10h / AH = 00h – Set Video Mode

INT 10h Service no. 00	Inputs any	Description	Example
AH=00 Set Video Mode	AL=00	Text mode 40×25, 16 color, 8 pages	
	AL=03	text mode. 80×25. 16 colors. 8 pages.	
	AL=13h	Graphical mode. 40×25. 256 colors. 320 x 200 pixels. 1 page.	MOV AL, 13h MOV AH, 0 INT 10h
AH=01h		Set text-mode cursor shape.	

Common Video Modes in 8086

Mode	Resolution	Colors	Memory per Screen	Pages Available (64KB VRAM)
03H (Text)	80×25	16 (4-bit)	4000 bytes (80×25×2)	8 pages
04H (Graphics)	320×200	4 (2-bit)	16 KB (320×200 / 4)	4 pages
06H (Graphics)	640×200	2 (1-bit)	16 KB (640×200 / 8)	4 pages
0DH (Graphics)	320×200	16 (4-bit)	32 KB (320×200×4/8)	2 pages
13H (Graphics)	320×200	256 (8-bit)	64 KB (320×200×1)	1 page

Text mode is when the screen shows characters (letters, numbers, symbols) — not individual pixels.
Graphics mode is when the screen is controlled pixel by pixel.

Common Video Modes in 8086

Mode	Resolution	Colors	Memory per Screen	Pages Available (64KB VRAM)
03H (Text)	80×25	16 (4-bit)	4000 bytes (80×25×2)	8 pages
04H (Graphics)	320×200	4 (2-bit)	16 KB (320×200 / 4)	4 pages
06H (Graphics)	640×200	2 (1-bit)	16 KB (640×200 / 8)	4 pages
0DH (Graphics)	320×200	16 (4-bit)	32 KB (320×200×4/8)	2 pages
13H (Graphics)	320×200	256 (8-bit)	64 KB (320×200×1)	1 page

A video page is a block of video memory that stores one complete screen display. Multiple pages allow several screens to exist in memory, but only one is shown at a time.

Text mode of 80 × 25 characters.

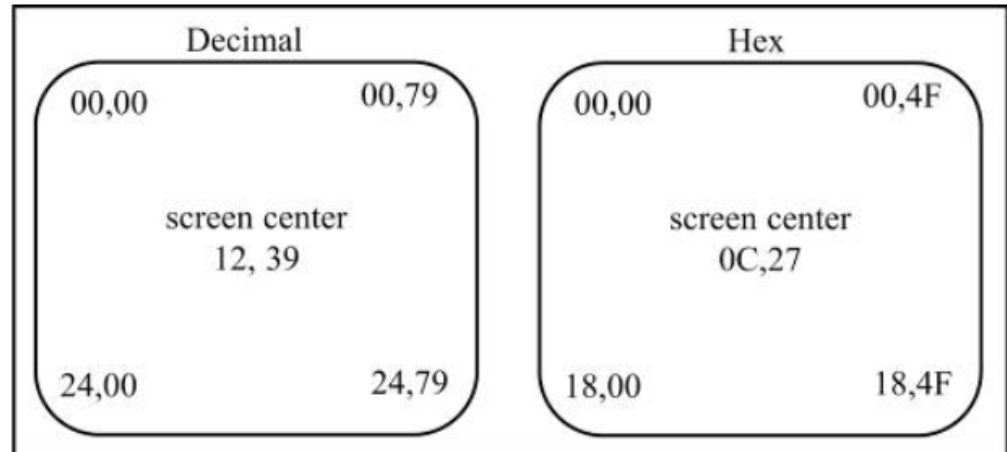
- A total of 2K ($80 \times 25 = 2000$) for characters, plus 2K for attributes, as each character has one attribute byte.
- In this mode, 16 colors are supported.
- To select this mode, use AL = 03 for mode selection in INT 10H option AH = 00.

Screen

The monitor screen in the x86 PC is divided into 80 columns and 25 rows in normal text mode.

- Columns are numbered from 0 to 79.
- Rows are numbered 0 to 24.

The top left corner has been assigned 00,00, the top right 00,79. Bottom left is 24,00, bottom right 24,79



Modes

INT 10h / AH = 01h – set text-mode cursor shape.

INT 10h Service no. 01h	Inputs any	Description	Example
AH=01h Set text mode cursor shape	CH	Cursor start line(bit 0-4) and option (bit 5-7) Cursor is visible if Bit 5 =0 else cursor is invisible	; hide Blinking text cursor: MOV CH, 32 MOV AH, 1 INT 10h ; show standard Blinking text cursor: MOV CH, 6 MOV CL, 7 MOV AH, 1 INT 10h show box-shaped Blinking text cursor: MOV CH, 0 MOV CL, 7 MOV AH, 1 INT 10h
	CL	Bottom cursor line(Bit 0-4)	

Modes

INT 10h / AH = 2 – Set cursor position.

INT 10h Service no.02h	Inputs any	Description	Example
AH=02h set cursor position.	DH = row. DL = column.		MOV DH, 10 MOV DH, 20 MOV BH, 0 MOV AH, 2 INT 10h
	BH = page number (0..7).		

Cursor Position

INT 10h / AH = 02h - set cursor position.

input:

DH = row.

DL = column.

BH = page number (0..7).

Write the code to set the cursor position to row = 15 = 0FH and column = 25 = 19H.

```
MOV AH,02  
MOV BH,00  
MOV DL,25  
MOV DH,15  
INT 10H
```

Set Cursor Position

```
.MODEL SMALL
.STACK 100H
.CODE
MAIN PROC
    MOV AH, 02H    ; Function to set cursor position
    MOV BH, 00H    ; Page number
    MOV DH, 10     ; Row (Y)
    MOV DL, 20     ; Column (X)
    INT 10H        ; Call BIOS Video Interrupt

    MOV AH, 4CH    ; DOS terminate function
    INT 21H
MAIN ENDP
END MAIN
```


Modes

INT 10h / AH = 08h – read Character and **attribute** at cursor position.

INT 10h Service no. 08h	Inputs any	Description
AH=08h read Character and attribute at cursor position	<i>input:</i> BH = page number. <i>return:</i> AH = attribute. AL = Character.	

Modes

INT 10h / AH = 09h – write Character and **attribute** at cursor position.

INT 10h Service no. 09h	Inputs any	Description
AH=09h; write Character and attribute at cursor position	<i>input</i> AL =Character to Display. BH =page number. BL =attribute. CX =number of times to write Character.	

Modes

INT 10h / AH = 0Ah -write Character only at cursor position.

INT 10h Service no. 0Ah	Inputs any	[
AH=0Ah write Character only at cursor position	<i>input:</i> AL = Character to Display. BH = page number. CX = number of times to write Character	

Print a Character at the Cursor Position

```
.MODEL SMALL
.STACK 100H
.CODE
MAIN PROC
    MOV AH, 09H    ; Function to display character
    MOV AL, 'A'    ; Character to print
    MOV BH, 00H    ; Page number
    MOV BL, 0FH    ; Attribute (White text)
    MOV CX, 1      ; Number of times to print
    INT 10H        ; Call BIOS Video Interrupt

    MOV AH, 4CH    ; DOS terminate function
    INT 21H
MAIN ENDP
END MAIN
```

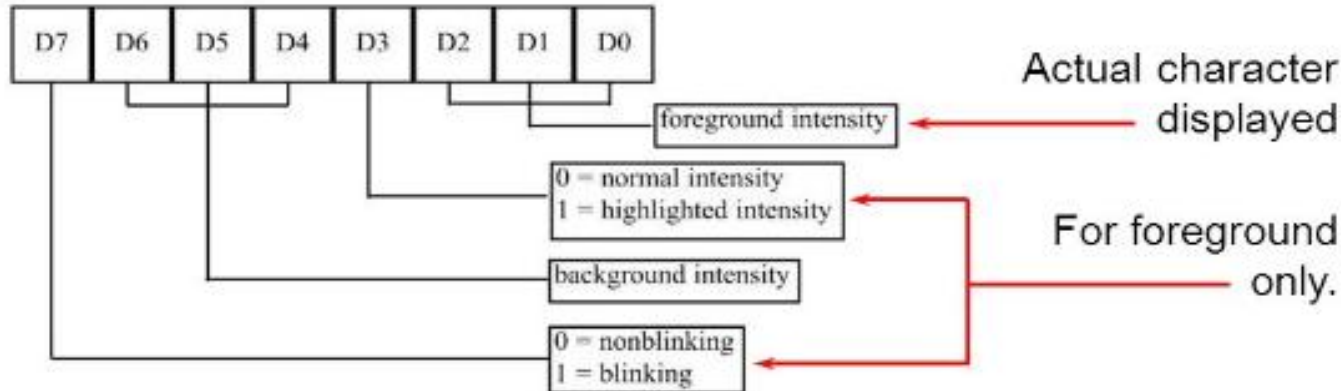
MASM Code to Clear Screen

```
.MODEL SMALL
.STACK 100H
.CODE
MAIN PROC
    MOV AH, 06H    ; Scroll up function
    MOV AL, 00H    ; Clear entire screen (0 means full scroll)
    MOV BH, 07H    ; Background: Black (0), Text color: Gray (7)
    MOV CH, 00H    ; Upper-left row (0)
    MOV CL, 00H    ; Upper-left column (0)
    MOV DH, 24H    ; Bottom-right row (24)
    MOV DL, 79H    ; Bottom-right column (79)
    INT 10H        ; Call BIOS Video Interrupt

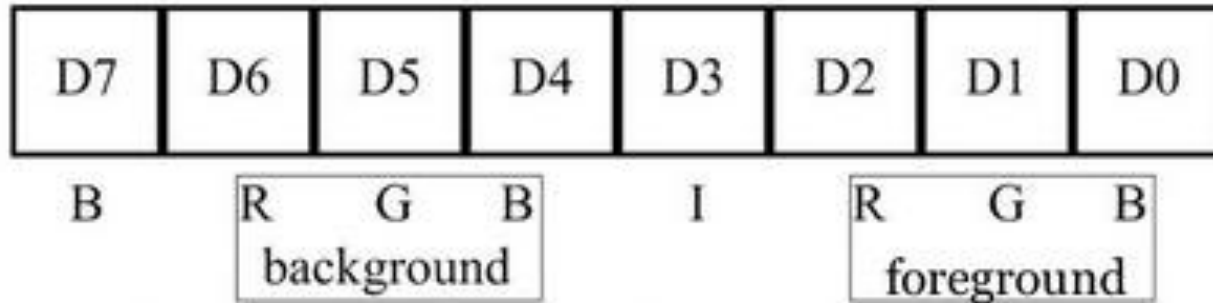
    MOV AH, 4CH    ; DOS terminate function
    INT 21H
MAIN ENDP
END MAIN
```

Attribute byte for Monochrome Monitors

- An attribute associated with each character on the screen provides information to the video circuitry.
 - Character (foreground) & background color/intensity.
- The attribute byte for each character on the monochrome monitor is limited.



Attribute byte in CGA mode



B = blinking I = foreground intensity

Blinking and intensity apply to foreground only.

Early hardware allowed only **8 background colors**. Hence only **3 bits for background**.
The **foreground** is the actual character you read, so it needs more visual options.

Colors

Character attribute is 8 bit value, low 4 bits set foreground color, high 4 bits set background color.
Background blinking not supported.

Table 4-1: The 16 Possible Colors

I	R	G	B	Color
0	0	0	0	black
0	0	0	1	blue
0	0	1	0	green
0	0	1	1	cyan
0	1	0	0	red
0	1	0	1	magenta
0	1	1	0	brown
0	1	1	1	white
1	0	0	0	gray
1	0	0	1	light blue
1	0	1	0	light green
1	0	1	1	light cyan
1	1	0	0	light red
1	1	0	1	light magenta
1	1	1	0	yellow
1	1	1	1	high intensity white

Some possible CGA colors and variations.

Binary	Hex	Color effect
0000 0000	00	Black on black
0000 0001	01	Blue on black
0001 0010	12	Green on blue
0001 0100	14	Red on blue
0001 1111	1F	High-intensity white on blue

E.g. 1000 0111
1111 0000

Example

Write a program using INT 10H to
b) Display the letter “D” in 200H locations with attributes black on white blinking(
blinking letters “D” are black and the screen background is white)

```
MOV AH, 09 ; Display Option
MOV BH, 00 ; Page 0
MOV AL, 44H ; ASCII Character for letter D
MOV CX, 200H; Repeat it 200H times
MOV BL, F0H; Black on white blinking
INT 10H
```

MOV BL, 1EH → Yellow text on
blue background

Modes

INT 10h / AH = 0CH – Change color for a Single pixel.

INT 10h Service no. 0Ch	Inputs any	Description	Example
AH=0CH Change color for a Single pixel	<i>input:</i> AL = pixel color CX = column. DX = row.		MOV AL, 13h MOV AH, 0 INT 10h; set graphics video mode. MOV AL, 1100b MOV CX, 10 MOV DX, 20 MOV AH, 0CH INT 10h; set pixel.

Modes

INT 10h / AH = 0Dh – get color of a Single pixel.

INT 10h Service no. 0Dh	Inputs any	Description	Example
AH=0Dh get color of a Single pixel	<i>input:</i> CX = column. DX = row. <i>output:</i> AL = pixel color		Â

Example

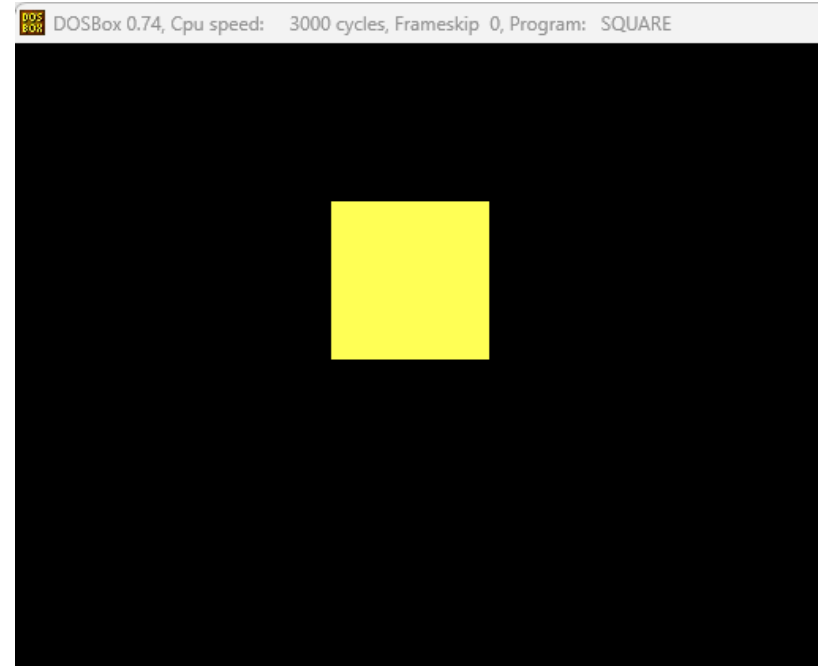
```
.MODEL SMALL
.STACK 100H
.CODE
MAIN PROC
    MOV AX, 0013H ; Set mode 13H (320x200 256-color)
    INT 10H

    MOV AH, 0CH ; Function to put a pixel
    MOV AL, 0FH ; Color (White)
    MOV CX, 100 ; X-coordinate
    MOV DX, 50 ; Y-coordinate
    INT 10H

    MOV AH, 4CH ; DOS terminate function
    INT 21H
MAIN ENDP
END MAIN
```

Drawing a Square

```
for y = 50 to 99  
  for x = 100 to 149  
    draw pixel(x,y)
```



Drawing a Square

```
MOV AX, 0013H
INT 10H
```

; Define square dimensions

```
MOV CX, 100 ; X-coordinate (starting position)
```

```
MOV DX, 50 ; Y-coordinate (starting position)
```

```
MOV SI, 50 ; Square size (width and height)
```

DRAW_ROW:

```
PUSH CX ; Save original X position
```

```
PUSH DX ; Save Y position
```

DRAW_COLUMN:

```
MOV AH, 0CH ; Function to put a pixel
```

```
MOV AL, 0EH ; Yellow color (0x0E)
```

```
MOV BH, 0 ; Video page number (always 0 in mode 13h)
```

```
INT 10H ; Call BIOS Video Interrupt
```

```
INC CX ; Move to the next column (right)
```

```
CMP CX, 150 ; Check if we've reached the right boundary (100 + 50)
```

```
JNE DRAW_COLUMN ; If not, keep drawing
```

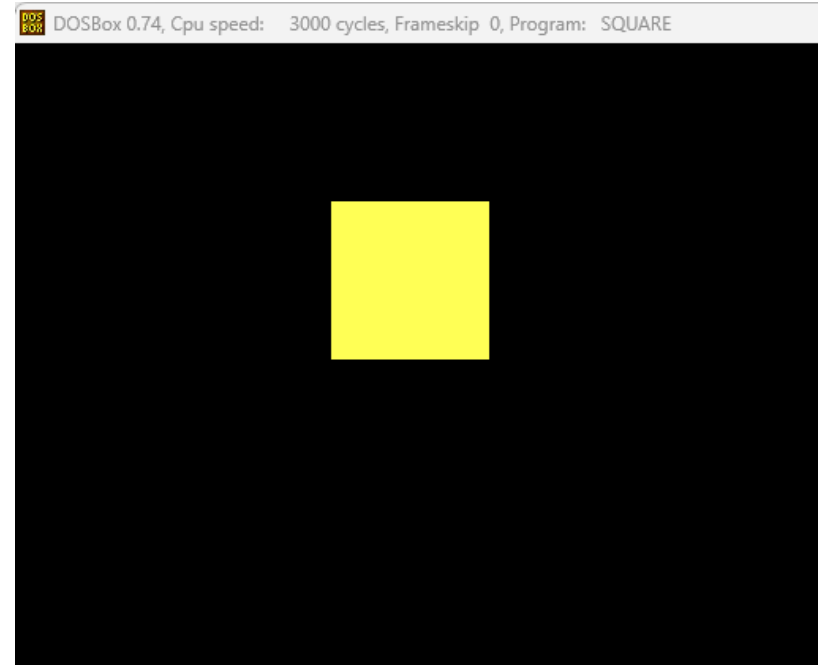
```
POP DX ; Restore Y position
```

```
POP CX ; Restore X position
```

```
INC DX ; Move to the next row (down)
```

```
CMP DX, 100 ; Check if we've reached the bottom boundary (50 + 50)
```

```
JNE DRAW_ROW ; If not, keep drawing
```



Summary

Function	Function code	Parameters	Return
Set video mode	AH=00h	AL = video mode ↗ AL = video mode (For Old IBM Only) ↗	AL = video mode flag / CRT controller mode byte
Set text-mode cursor shape	AH=01h	CH = Scan Row Start, CL = Scan Row End Normally a character cell has 8 scan lines, 0–7. So, CX=0607h is a normal underline cursor, CX=0007h is a full-block cursor. If bit 5 of CH is set, that often means "Hide cursor". So CX=2607h is an invisible cursor. Some video cards have 16 scan lines, 00h-0Fh. Some video cards don't use bit 5 of CH. With these, make Start>End (e.g. CX=0706h)	
Set cursor position	AH=02h	BH = Page Number, DH = Row, DL = Column	
Get cursor position and shape	AH=03h	BH = Page Number	AX = 0, CH = Start scan line, CL = End scan line, DH = Row, DL = Column

Summary

Select active display page	AH=05h	AL = Page Number	
Scroll up window	AH=06h	AL = lines to scroll (0 = clear, CH, CL, DH, DL are used), BH = Background Color and Foreground color. BH = 43h, means that background color is red and foreground color is cyan. Refer the BIOS color attributes CH = Upper row number, CL = Left column number, DH = Lower row number, DL = Right column number	
Scroll down window	AH=07h	like above	
Read character and attribute at cursor position	AH=08h	BH = Page Number	AH = Color , AL = Character
Write character and attribute at cursor position	AH=09h	AL = Character, BH = Page Number, BL = Color , CX = Number of times to print character	
Write character only at cursor position	AH=0Ah	AL = Character, BH = Page Number, CX = Number of times to print character	
Set background/border color	AH=0Bh, BH = 00h	BL = Background/Border color (border only in text modes)	

Summary

Set palette	AH=0Bh, BH = 01h	BL = Palette ID (was only valid in CGA , but newer cards support it in many or all graphics modes)	
Write graphics pixel	AH=0Ch	AL = Color , BH = Page Number, CX = x, DX = y	
Read graphics pixel	AH=0Dh	BH = Page Number, CX = x, DX = y	AL = Color
Teletype output	AH=0Eh	AL = Character, BH = Page Number, BL = Color (only in graphic mode)	
Get current video mode	AH=0Fh		AL = Video Mode, AH = number of character columns, BH = active page
Change text mode character set ^[3]	AH=11h	BH = Number of bytes per character, CX = Number of characters to change, DX = Starting character to change, ES:BP = Offset of character data	
Write string (EGA+, meaning PC AT minimum)	AH=13h	AL = Write mode, BH = Page Number, BL = Color , CX = Number of characters in string, DH = Row, DL = Column, ES:BP = Offset of string	

References

https://www.ic.unicamp.br/~celio/mc404-2004/service_interrupts#attrib



BITS Pilani
K K Birla Goa Campus

THANK YOU

